



IN410

Cryptography and Secure Communications

Ahmad Fadlallah



Course Syllabus

- Context
- History/ Classical ciphers
- Symmetric Encryption
- Cryptographic hash functions
- Asymmetric Encryption
- Key Distribution
- Authentication



Credits

- The following slides are the adaptation of
 - Lecture slides of Lawrie Brown based on William Stallings book “Cryptography and Network Security: Principles and Practice”
 - Lecture Slides of Dan Boneh – Stanford University
 - Lecture slides of Pawel Wocjan – University of Central California
 - Lecture Slides of Ahmed Serhrouchni – Telecom ParisTech
 - Others (References in the note section)



Cryptographic Hash Functions



Outline

- Introduction
- Unkeyed Hash Functions
- Message Authentication Codes
- Attacks on Hash Functions
- Key Derivation Functions



Introduction

Unkeyed Hash Functions

Message Authentication Codes

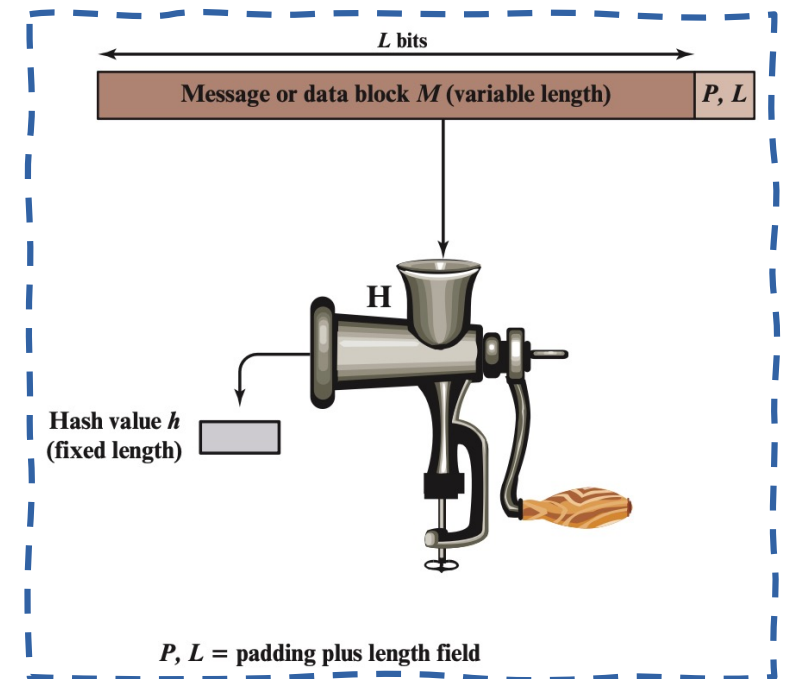
Attacks on Hash Functions

Key Derivation Functions



Definitions

- A **hash function** H accepts a variable-length block of data M as input and produces a fixed-size **hash value**: $h = H(M)$
 - h is the **digest** of M
 - M is the **preimage** of h
- Message padding might apply





General Hash Functions Properties

- General Hash Function Properties
 - **Compression:** by definition
 - **Ease of computation:** Given an input M , the hash value $H(M)$ of M is easy to compute



Cryptographic Hash Functions Properties

One-way hash function (pre-image resistance): given a hash value h (for which no pre-image is known), it is computationally infeasible to find **any** input $M \mid H(M) = h$

???



h

Weak collision resistance (2nd pre-image resistance): given an input M , it is computationally infeasible to find **any** second input $M' \neq M \mid H(M') = H(M)$

M



???



h

Strong collision resistance (collision resistance): it is computationally infeasible to find **any** two distinct inputs $M \neq M' \mid H(M) = H(M')$

???



???

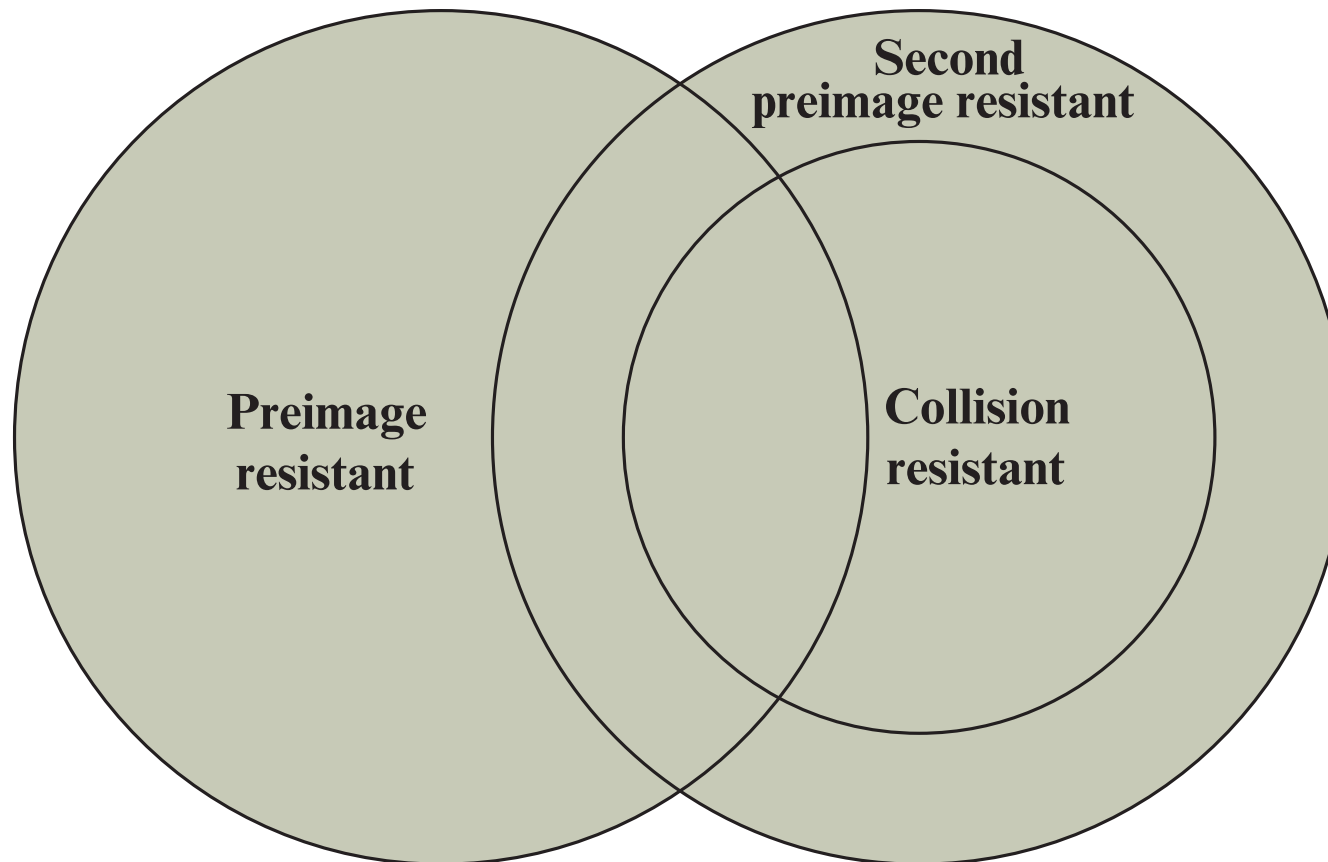


h

Pseudo-randomness



Cryptographic Hash Functions Properties





Exercise

A cryptographic hash function h takes as input a message of arbitrary length and produces as output a message digest of fixed length.

Certain properties should be however satisfied:

- a. Ease of Computation:** Given a message m , $h(m)$ can be calculated very quickly.
- b. One-Way hash function:** Given a message digest y , it is computationally infeasible to find an m with $h(m) = y$
- c. Strong Collision Resistance:** It is computationally infeasible to find two different messages m_1 and m_2 with $h(m_1) = h(m_2)$

Let n be a large integer. Let $h(m) = m \pmod n$

Check if $h(m)$ satisfies (or not) the properties a, b and c



Exercise

Let the input data be of the form

$X = (X_0, X_1, X_2, \dots, X_{n-1})$, where each X_i is a byte

Consider the following hash function :

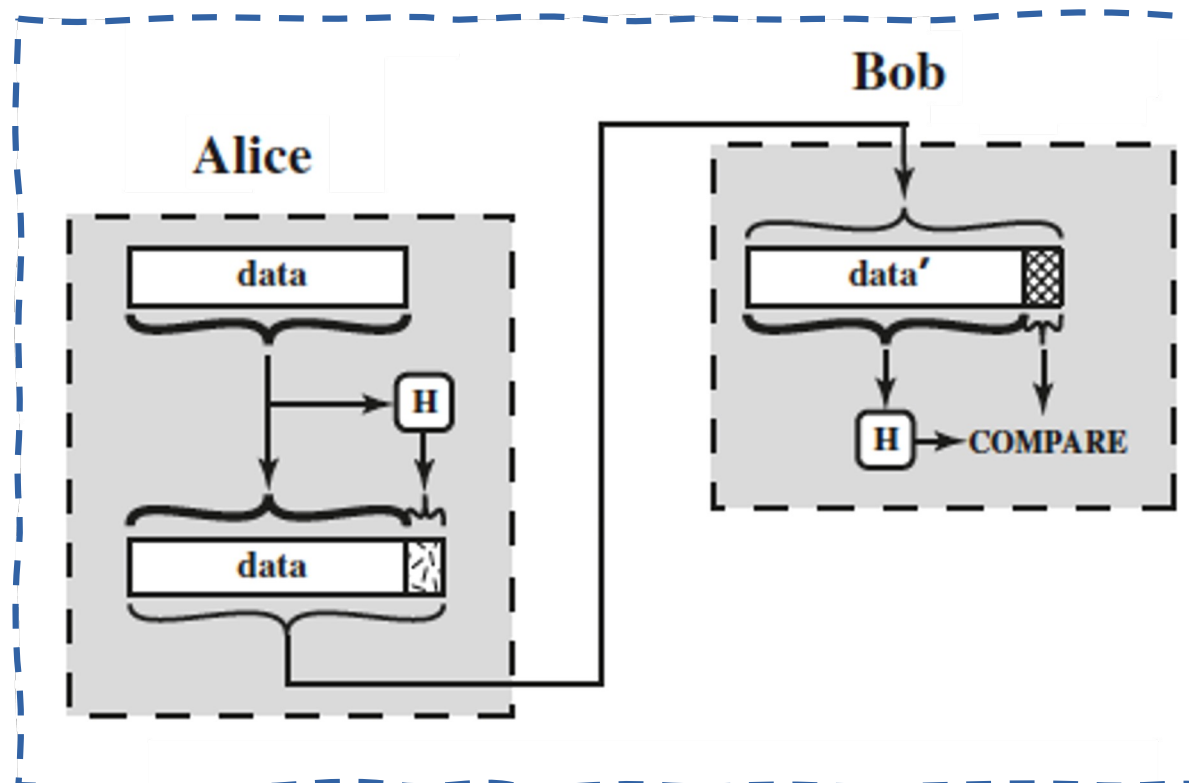
$$h(X) = X_0 \oplus X_1 \oplus X_2 \oplus \dots \oplus X_{n-1}$$

Is this a secure hashing method in the sense that collisions are hard to find ?



Hash Functions Applications

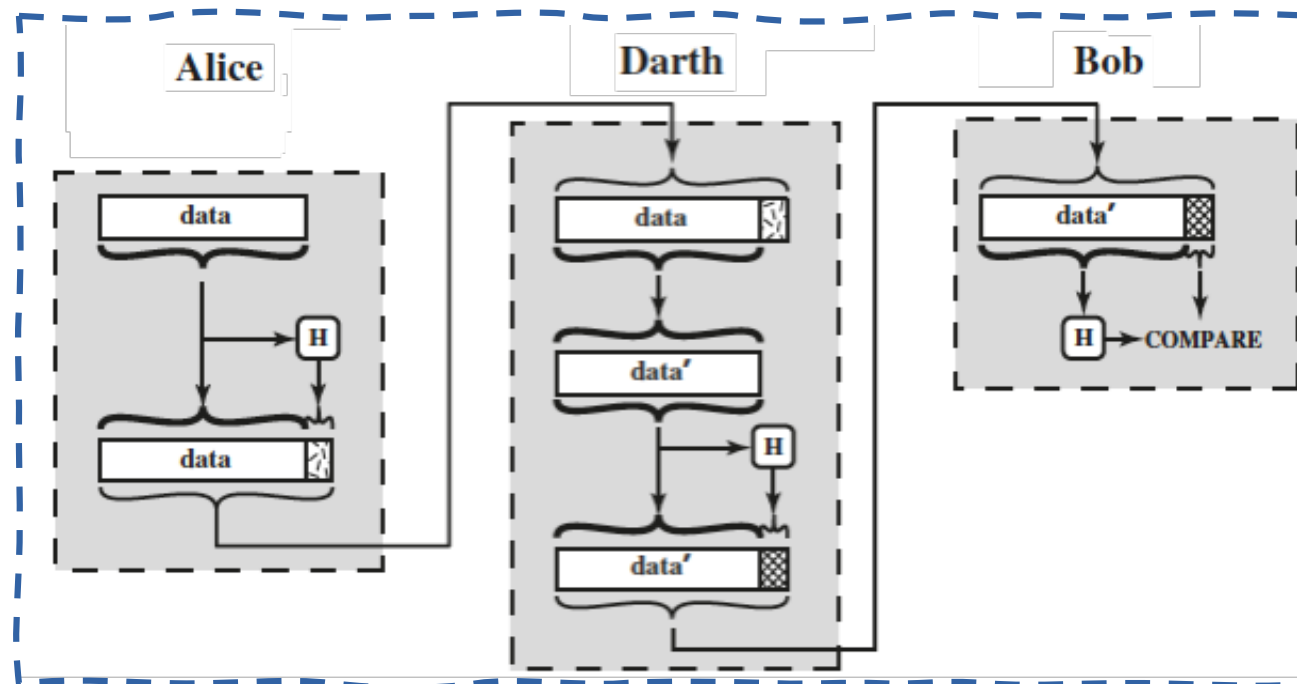
- The principal objective of a hash function is **data integrity**





Hash Functions Applications

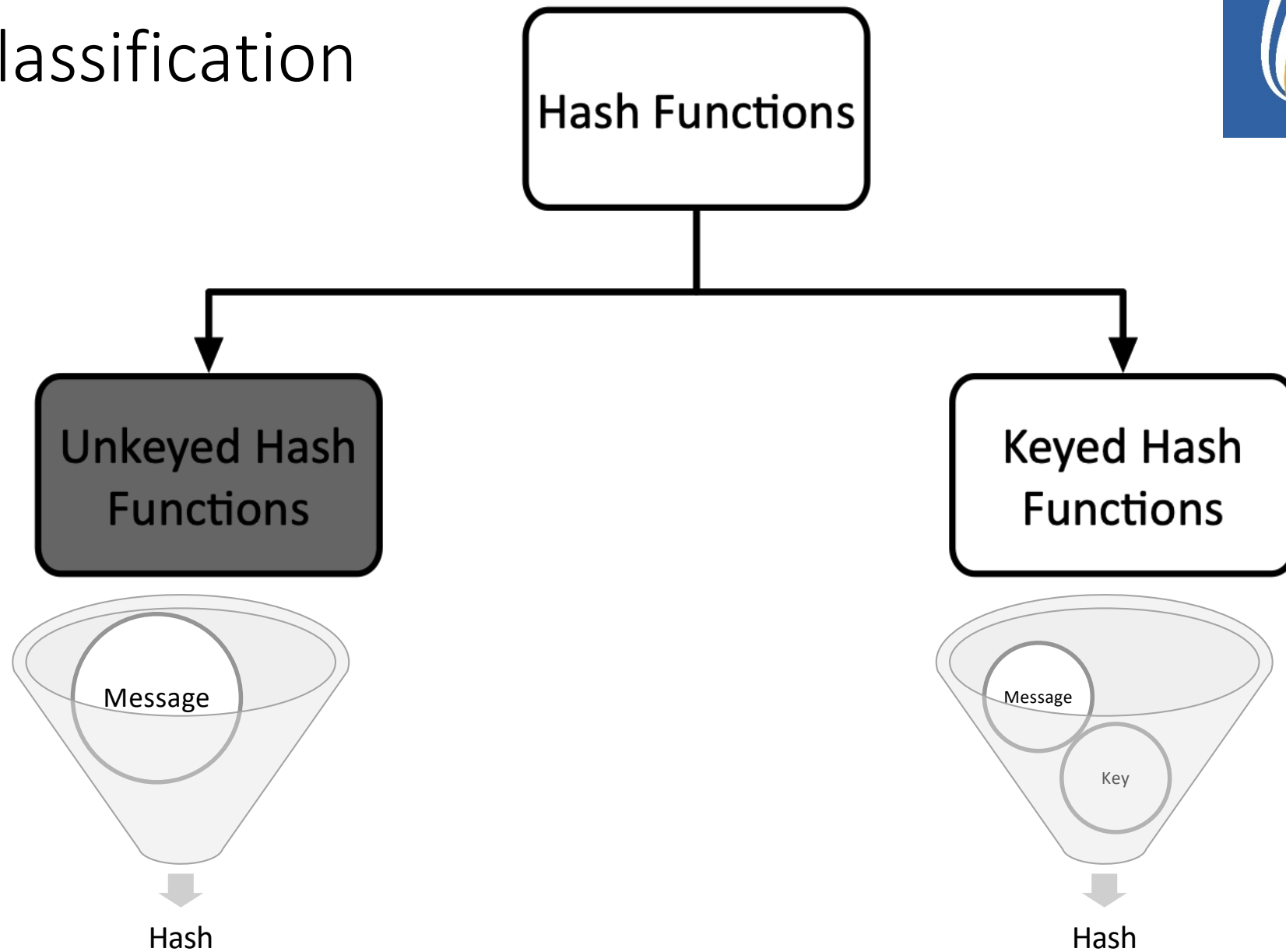
- Hash functions can detect **random but not malicious errors**



- Integrity requires a secret key



Classification





Introduction

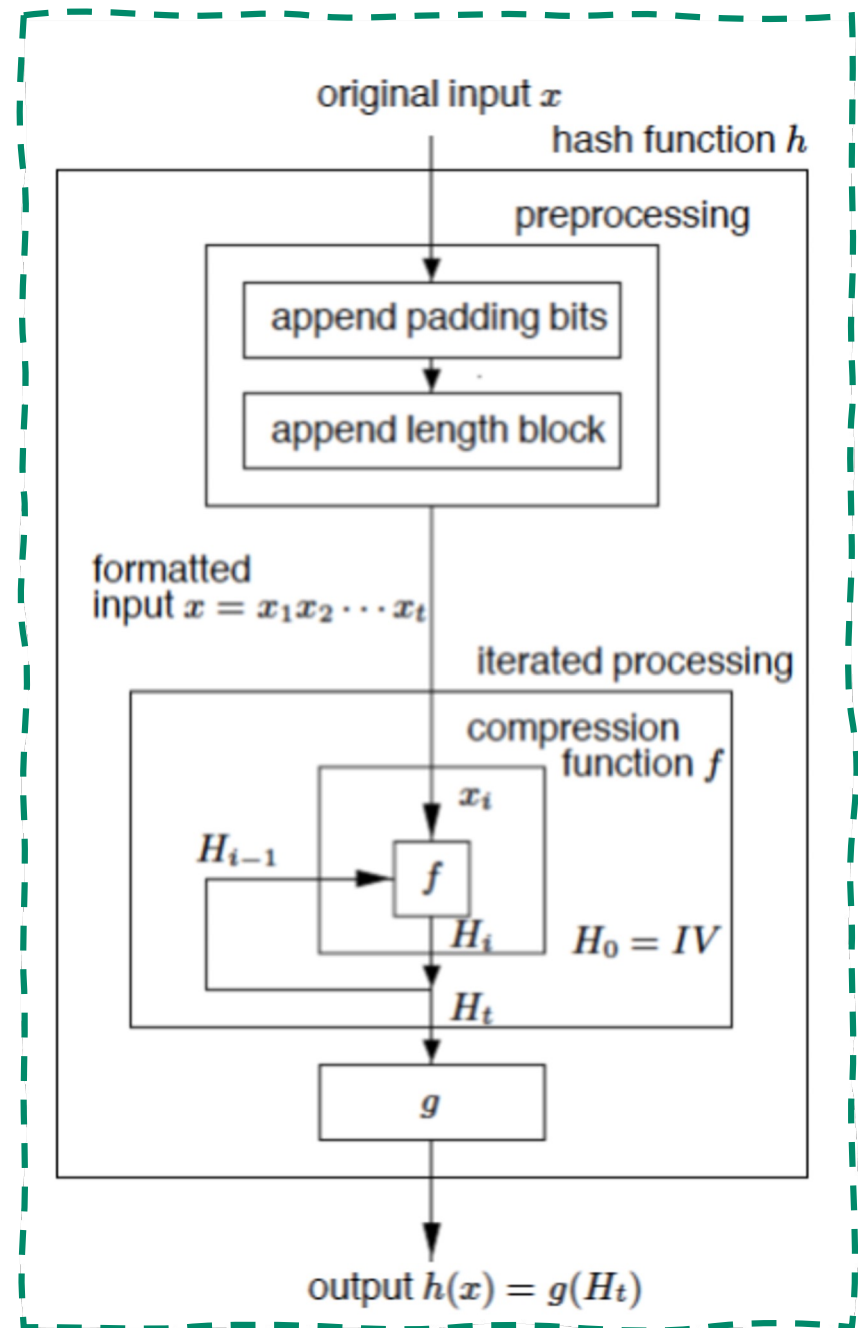
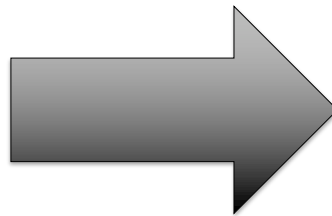
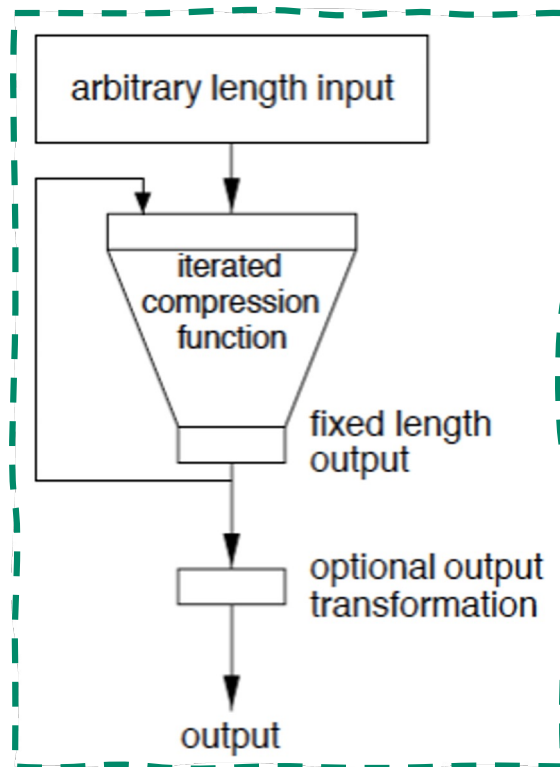
Unkeyed Hash Functions

Message Authentication Codes

Attacks on Hash Functions

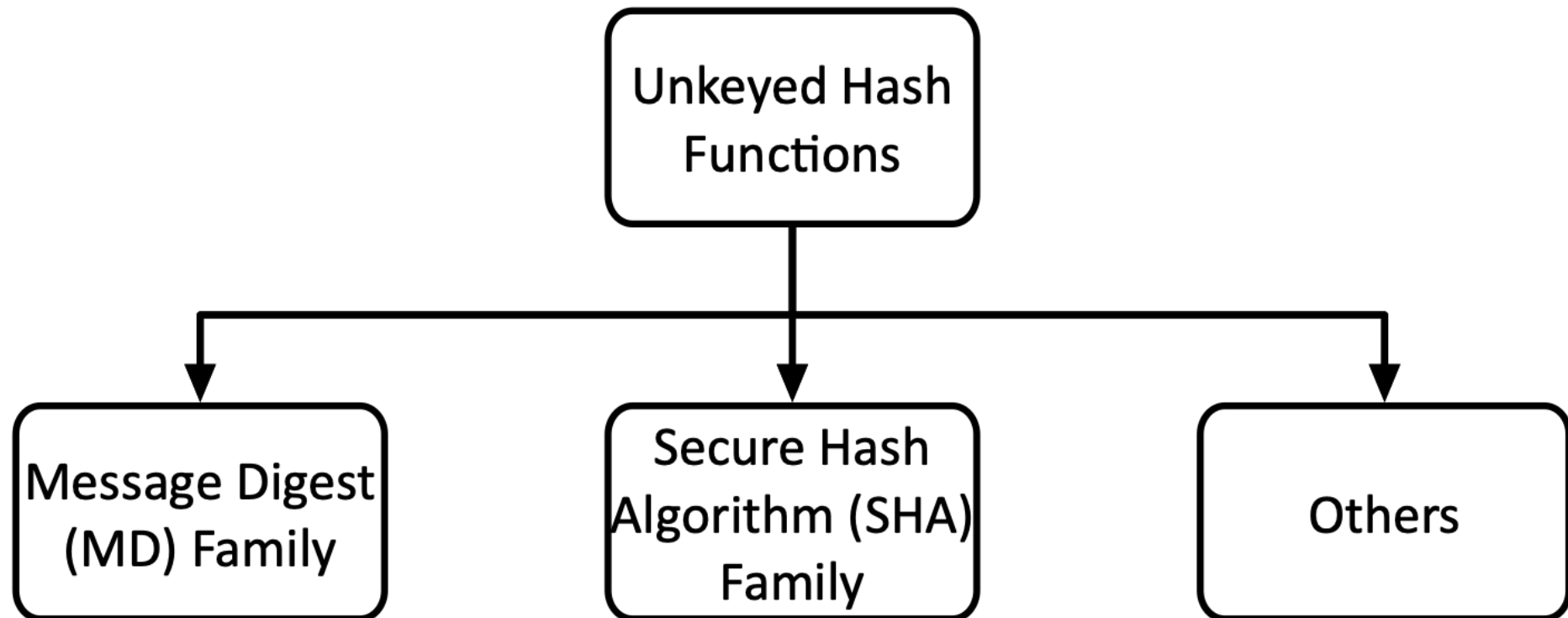
Key Derivation Functions

Unkeyed Hash Functions





Unkeyed Hash functions

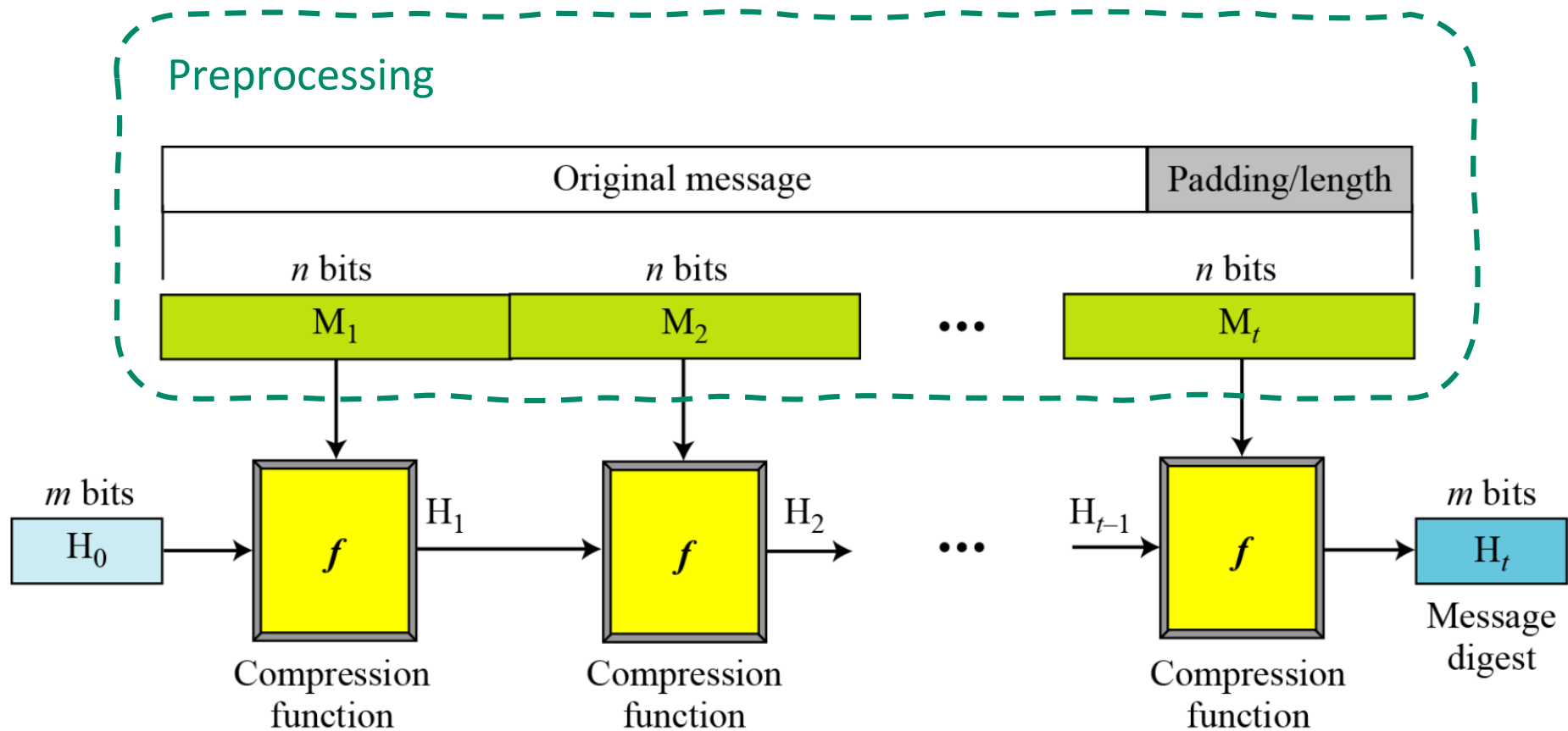




Unkeyed Hash Functions: MD5

- Member of the **Message Digest** Cryptographic Hash functions family (MD2, MD4, MD5 and MD6)
- Developed by *Ronald Rivest*
- First published in April 1992 (RFC 1321)
- **Digest size**: 128 bit
- Uses the **Merkle–Damgård** construction
- Block size: 512 bits
- Number of Rounds: 4

MD5: Merkle-Damgård Construction





MD5 Overview - Preprocessing

- Pad message so its length is $\equiv \underline{448 \bmod 512}$
- Append a 64-bit length value to message

MD5 Overview - Preprocessing



- **Message to hash:** “abc”
- **Binary message:** 01100001 01100010 01100011
- **Total length:** 24 bits
- **Padding method:** add a single bit (1) and the rest of padding is (0)s
- **Padding:** 01100001 01100010 01100011 100000 ... 0 (total 448 bits)
- **Append the length field (64 bits):** Original message (before padding) length in bits
 - Original message length = 24 bits.
 - Over 64 bits: $(24)_{10} = 00000000 \dots [56 \text{ bits}]00011000$

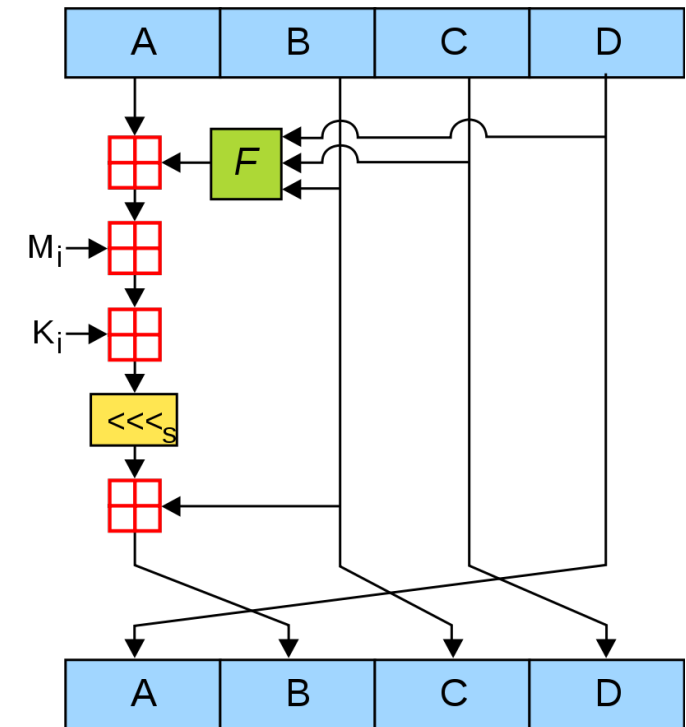
- Final preprocessed message:

01100001 01100010 01100011 10000000 00 ... [448 bits] ...
... 00000000 00000000 00000000 00011000



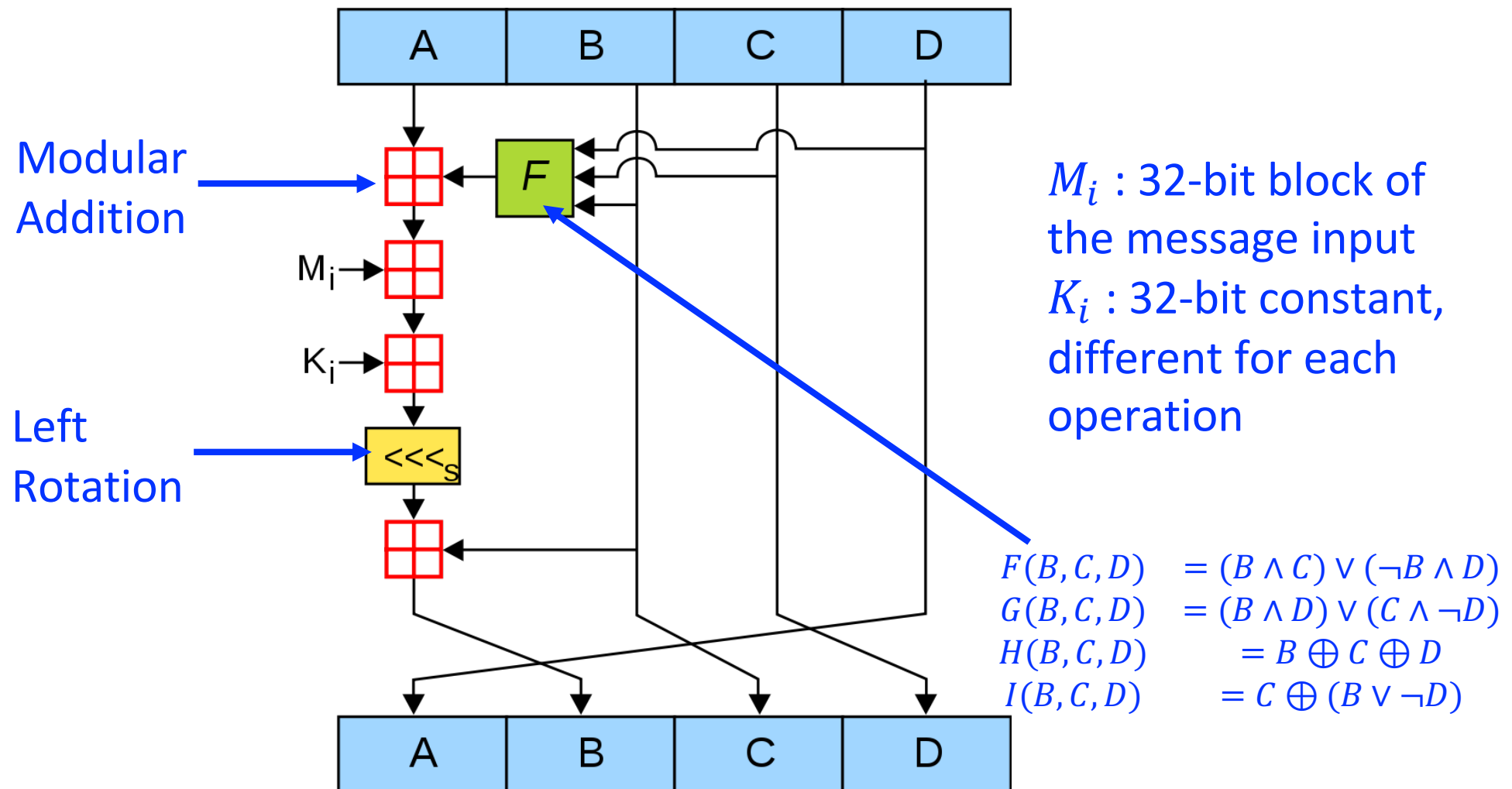
MD5 Overview - Hashing

- Initialize 4-word (128-bit) MD buffer (A,B,C,D)
- Process message in 512-bit blocks:
 - Each block is split into 16 words (4B)
 - Using 4 rounds of 16 operations on message block & buffer
 - One word from the message block is processed in each round
 - Add output to buffer input to form new buffer value
- Output hash value is the final buffer value





MD5 – Round Operation





MD5 - Security

- Several attacks on MD5
- Proven to be **"Not Collision-Resistant"**

Cannot be used as a cryptographic hash function





MD5 – Collision Example

```
d131dd02c5e6eec4 693d9a0698aff95c 2fcab58712467eab 4004583eb8fb7f89  
55ad340609f4b302 83e488832571415a 085125e8f7cdc99f d91dbdf280373c5b  
d8823e3156348f5b ae6dacd436c919c6 dd53e2b487da03fd 02396306d248cda0  
e99f33420f577ee8 ce54b67080a80dle c69821bcb6a88393 96f9652b6ff72a70
```

```
d131dd02c5e6eec4 693d9a0698aff95c 2fcab50712467eab 4004583eb8fb7f89  
55ad340609f4b302 83e4888325f1415a 085125e8f7cdc99f d91dbd7280373c5b  
d8823e3156348f5b ae6dacd436c919c6 dd53e23487da03fd 02396306d248cda0  
e99f33420f577ee8 ce54b67080280dle c69821bcb6a88393 96f965ab6ff72a70
```

Secure Hash Algorithms (SHA)



SHA-0

- 160-bit hash function published in 1993 under the name "SHA".
- **Withdrawn** due to design flaws

SHA-1

- A 160-bit hash function similar to the MD5 algorithm.
- Designed by the National Security Agency (NSA)
- Cryptographic weaknesses were discovered in SHA-1, and the standard was no longer approved for most cryptographic uses after 2010.

Secure Hash Algorithms (SHA)



SHA-2

- A family of two similar hash functions, with different block sizes, known as **SHA-256** and **SHA-512**.
- They differ in the word size; **SHA-256** uses 32-bit words where **SHA-512** uses 64-bit words.
- There are also truncated versions of each standard, known as SHA-224 and SHA-384, also designed by the NSA.

SHA-3

- A hash function formerly called **Keccak**, chosen in 2012 after a public competition among non-NSA designers.
- It supports the same hash lengths as SHA-2, and its internal structure **differs significantly** from the rest of the SHA family.

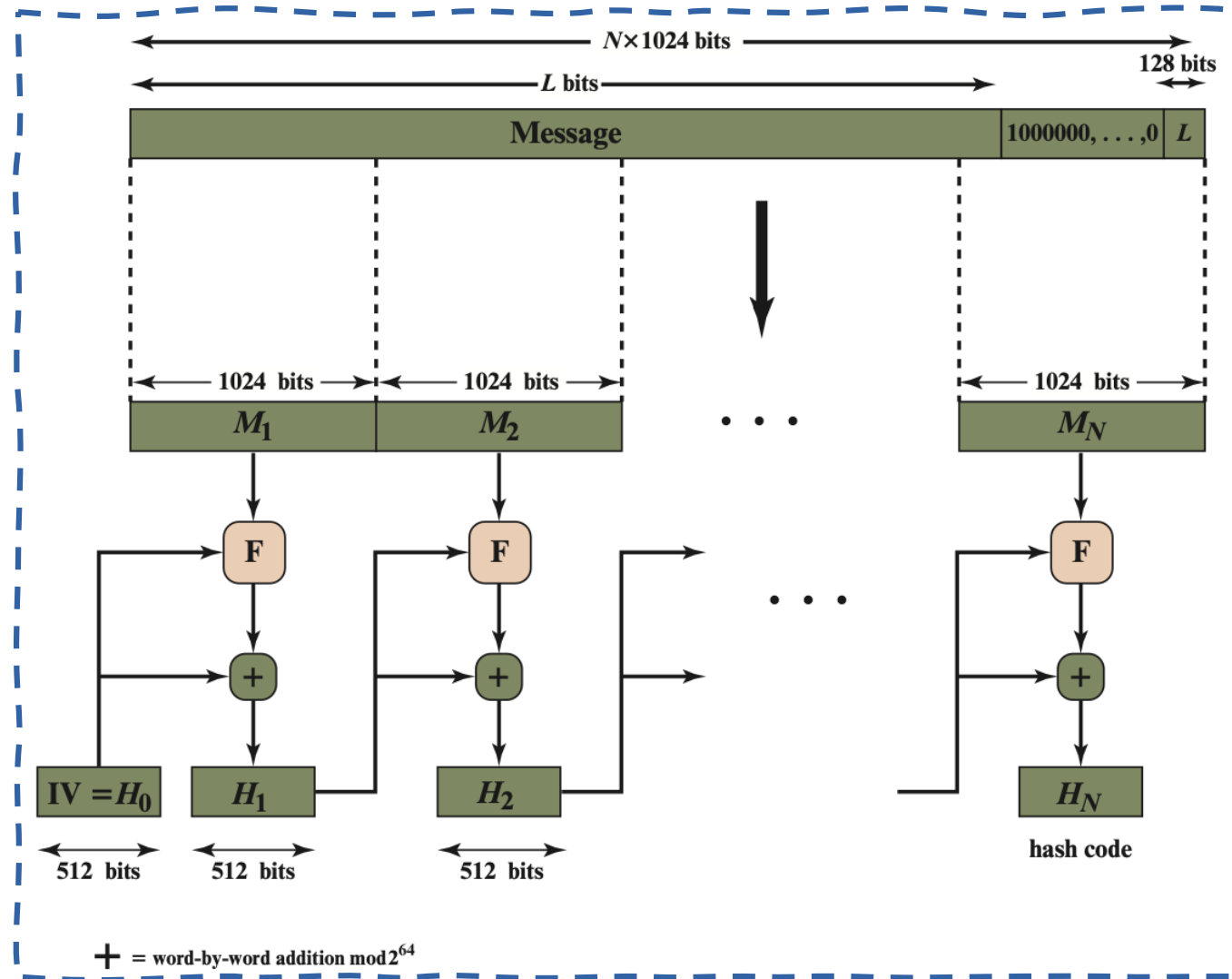


SHA-2 versions

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message digest size	160	224	256	384	512
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block size	512	512	512	1024	1024
Word size	32	32	32	64	64
Number of steps	80	64	64	80	80



SHA-512 : Overview



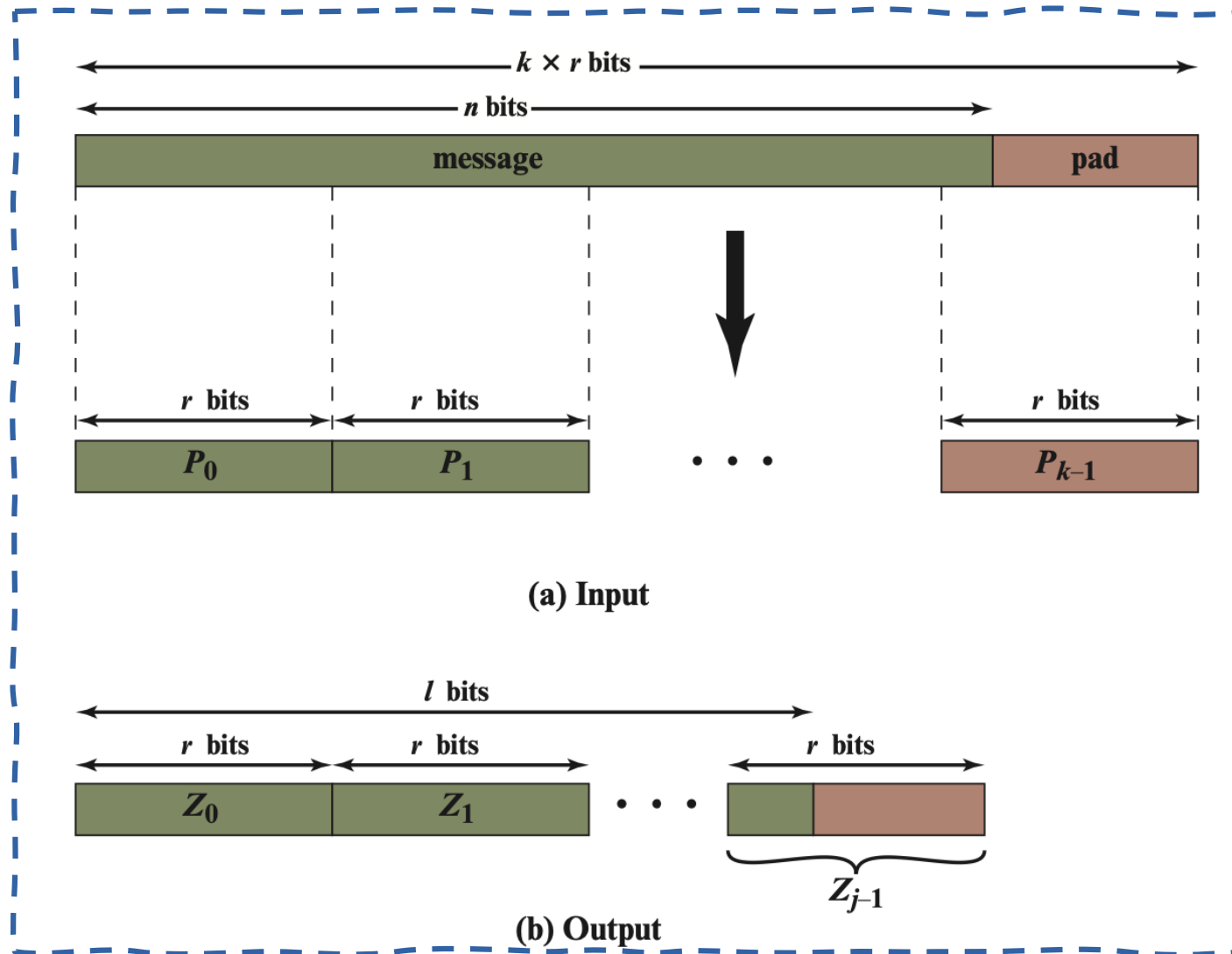


SHA-3 Requirements

- Replace SHA-2 with SHA-3 in any use
 - Use same hash sizes
- Preserve the online nature of SHA-2
 - So must process small blocks (512/1024 bits) instead of requiring that the entire message be buffered in memory before processing it

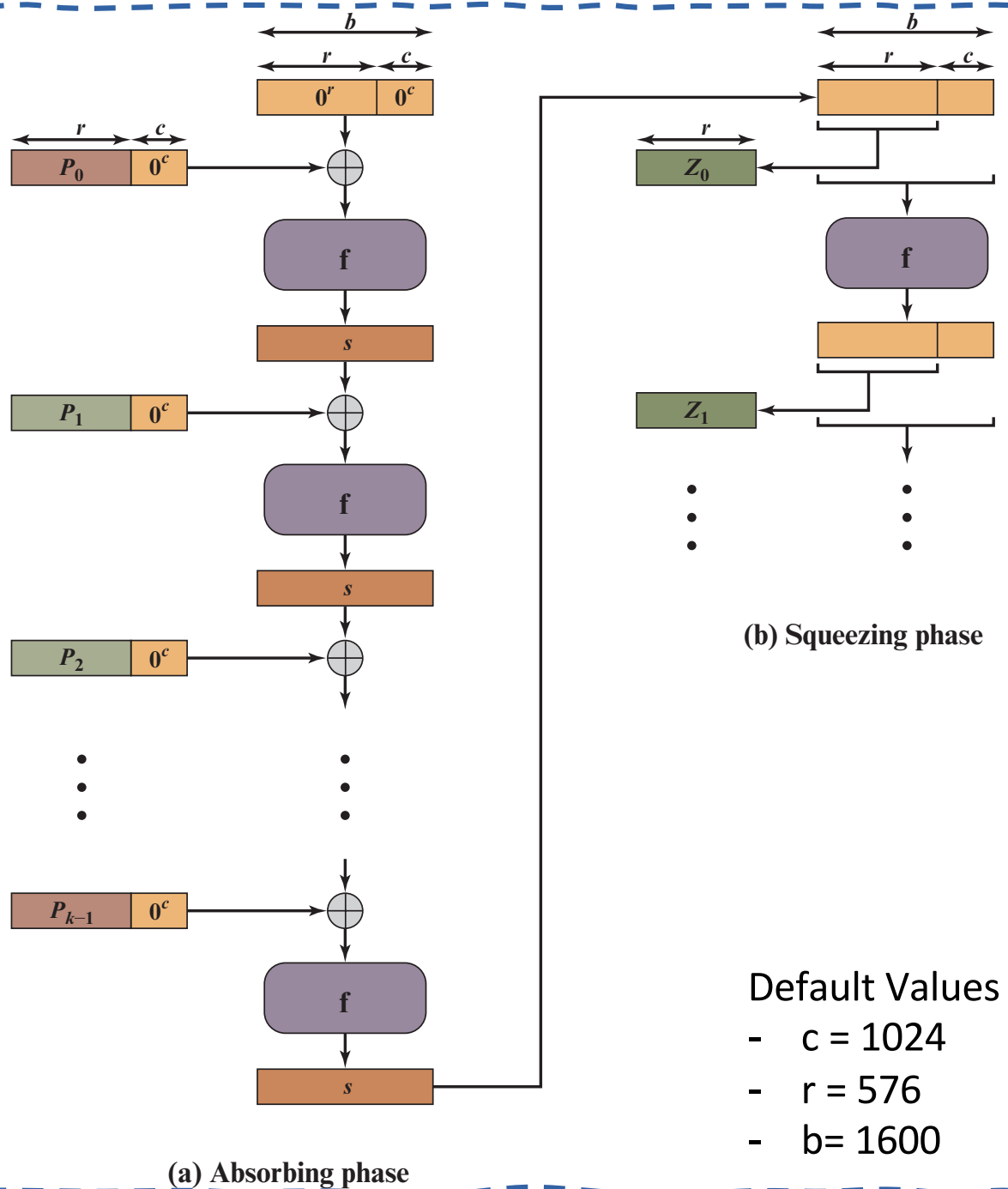


SHA-3 Details (cont'd)





Sponge Function





Introduction

Unkeyed Hash Functions

Message Authentication Codes

Attacks on Hash Functions

Key Derivation Functions

Message Authentication Code (MAC)



- **Goal**: integrity, no confidentiality
- Examples:
 - Protecting public binaries on disk
 - Protecting banner ads on web pages



Message Authentication Code (MAC)

Generated by an algorithm that creates a **small fixed-sized block**

- Depending on both message and some key
- Like encryption though need not be reversible

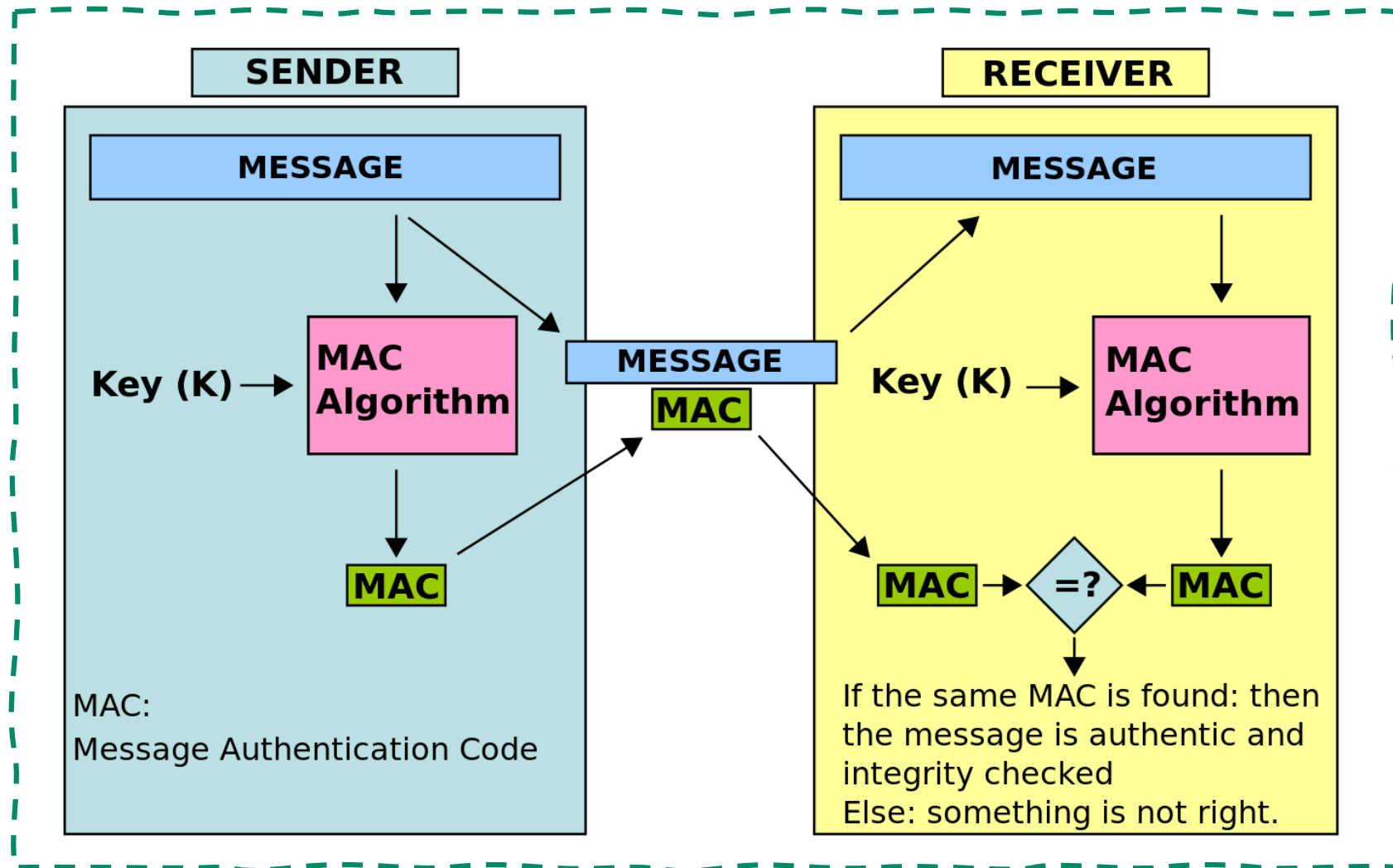
Appended to message as a **signature**

Receiver performs same computation on message and checks it matches the MAC

Provides assurance that message is unaltered and comes from sender

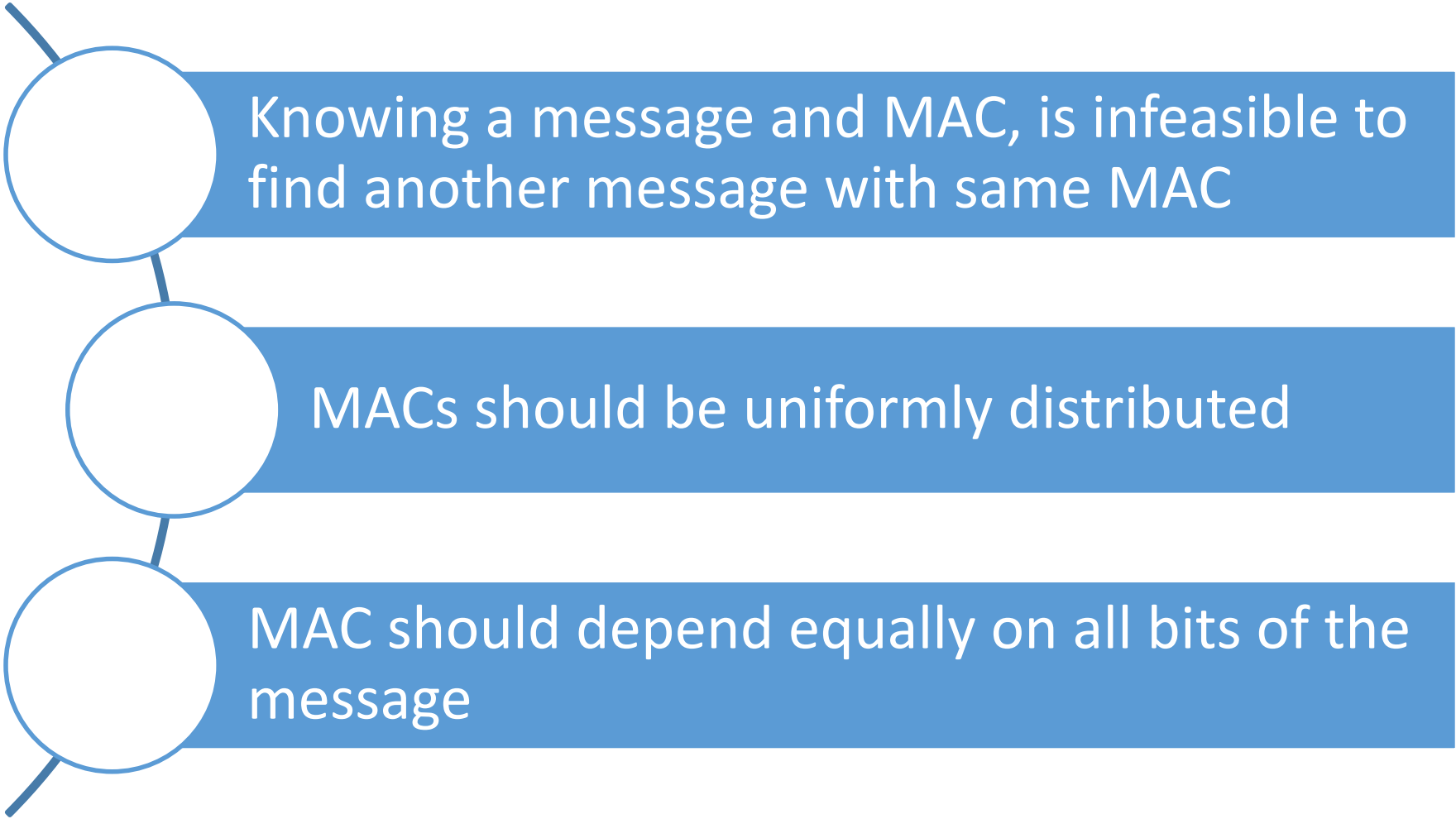


Message Authentication Code (MAC)





Requirements for MACs

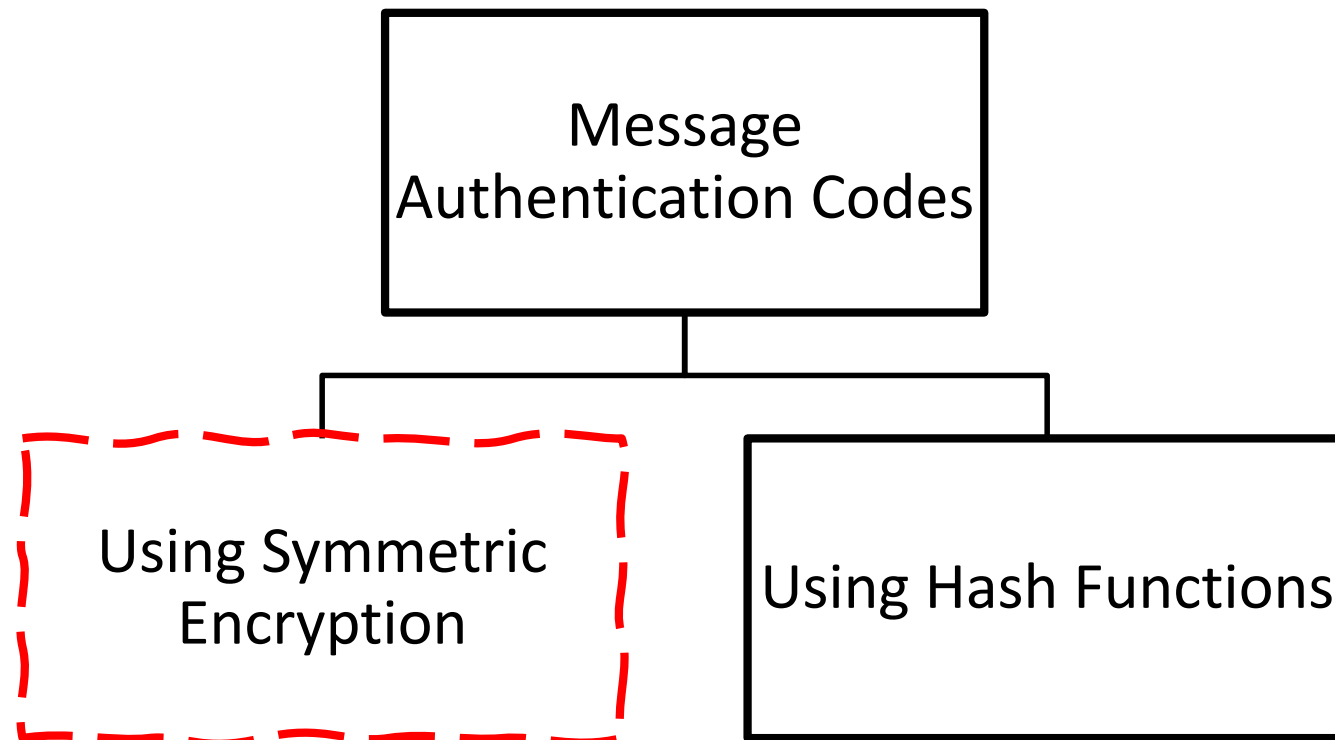


Knowing a message and MAC, is infeasible to find another message with same MAC

MACs should be uniformly distributed

MAC should depend equally on all bits of the message

Message Authentication Code (MAC) Classification



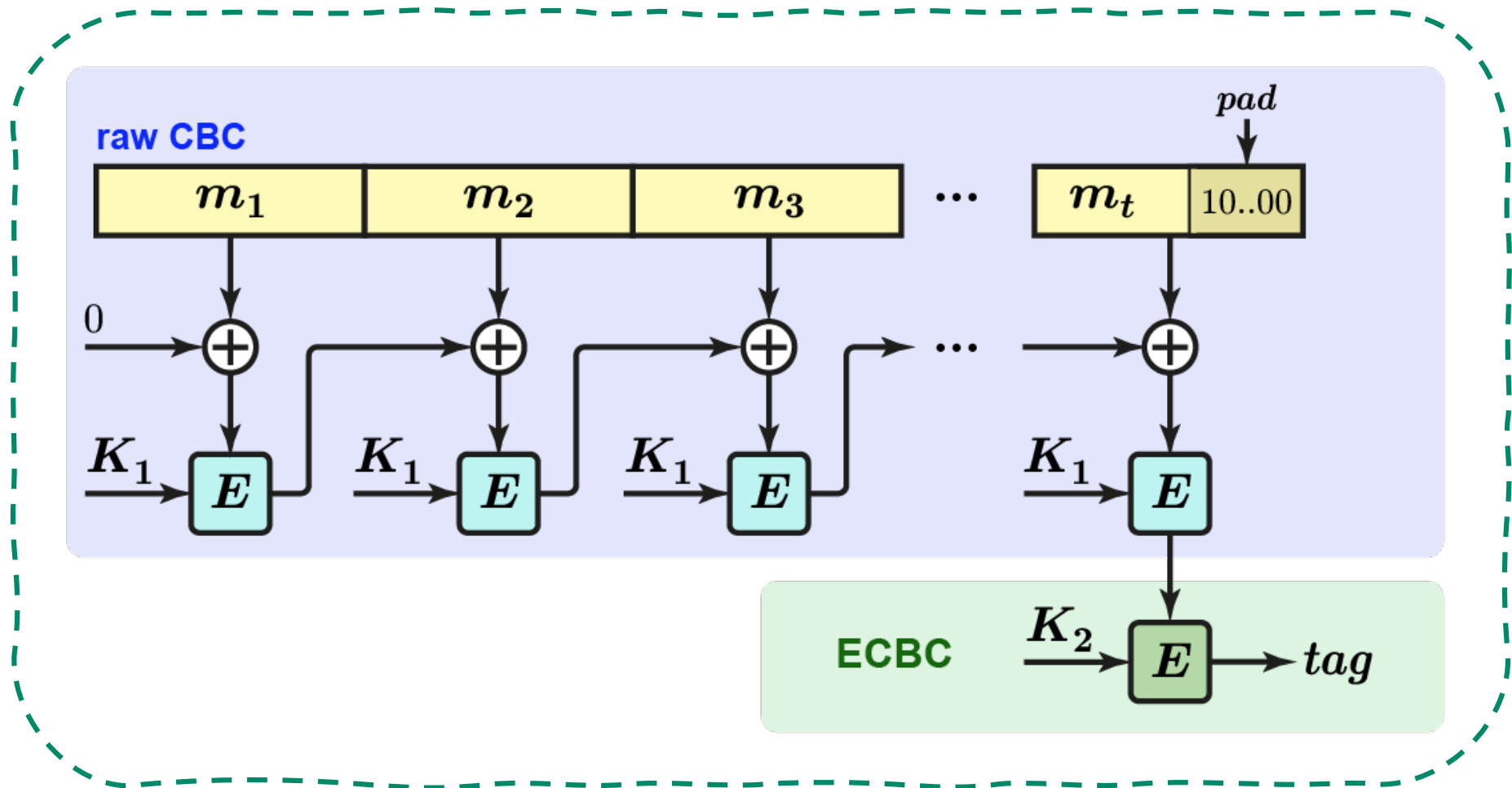


Using Symmetric Ciphers for MACs

- Can use any Block cipher Chaining mode and use final block as a MAC
- Different constructions
 - **Encrypted CBC-MAC**
 - **Nested MAC**
 - ...

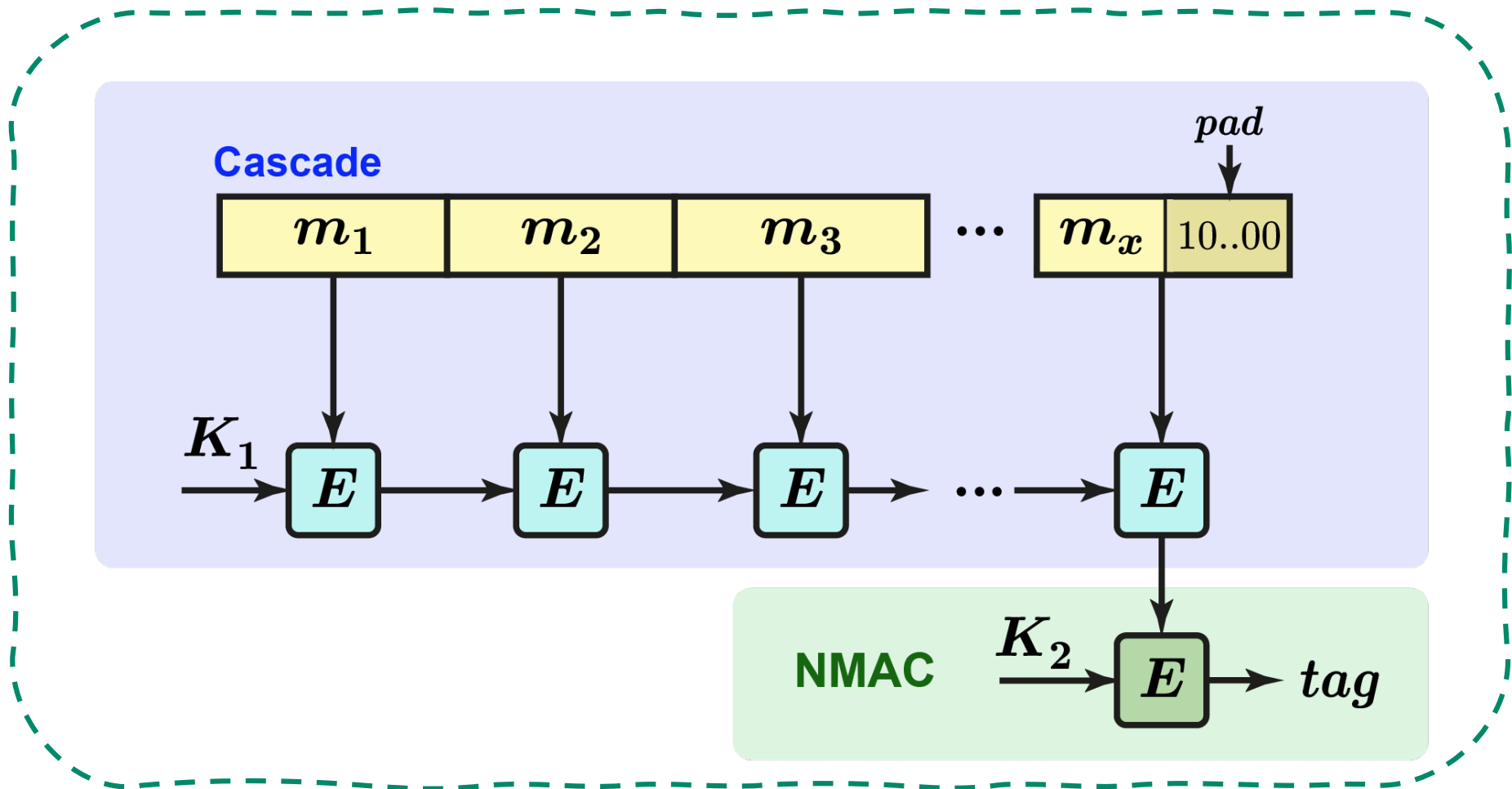


Encrypted CBC-MAC

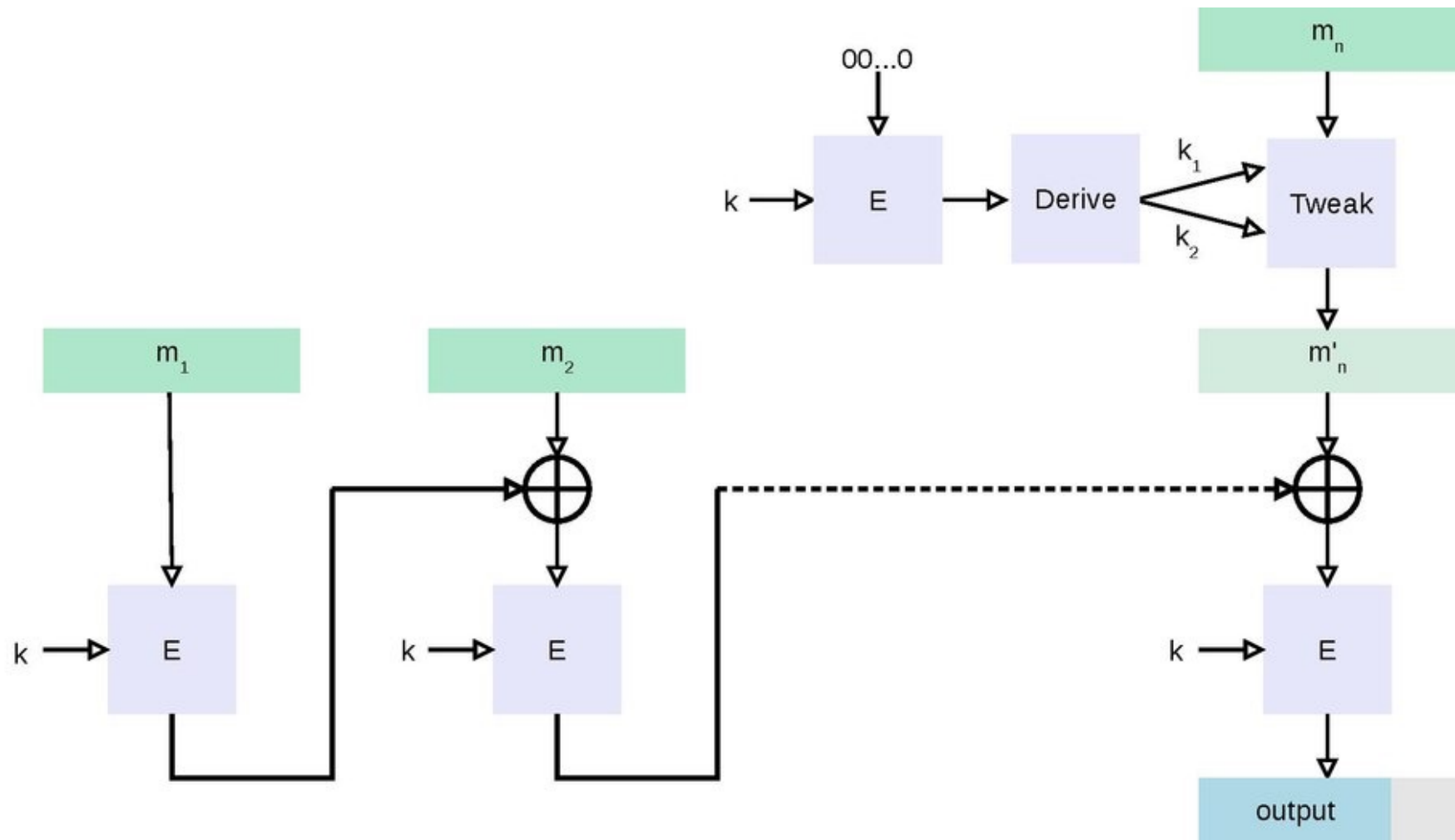




Nested MAC



NIST CMAC



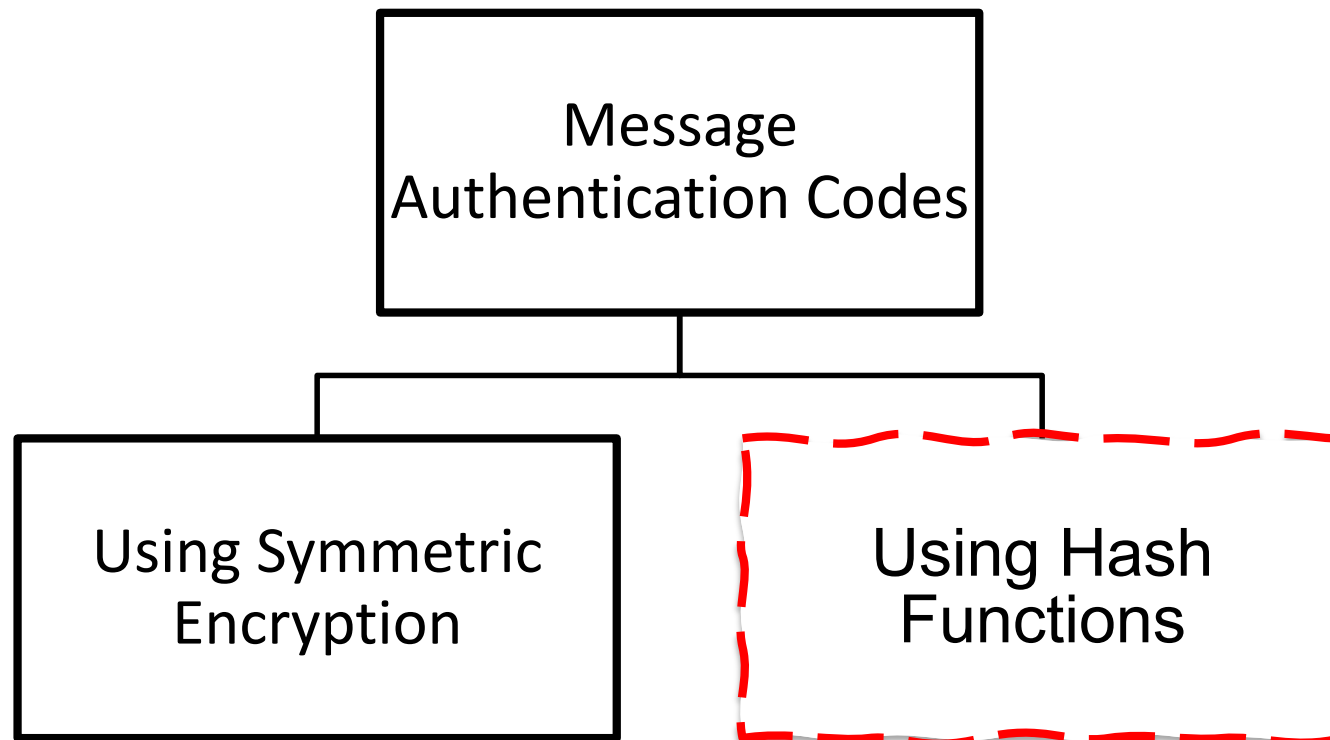


Other MAC Constructions

- CMAC (NIST standard)
 - Variant of CBC-MAC using multiple keys
- PMAC (Parallelizable MAC)
- GMAC (Galois-Counter Mode MAC)
- ...



MAC Classification





Keyed Hash Functions as MACs

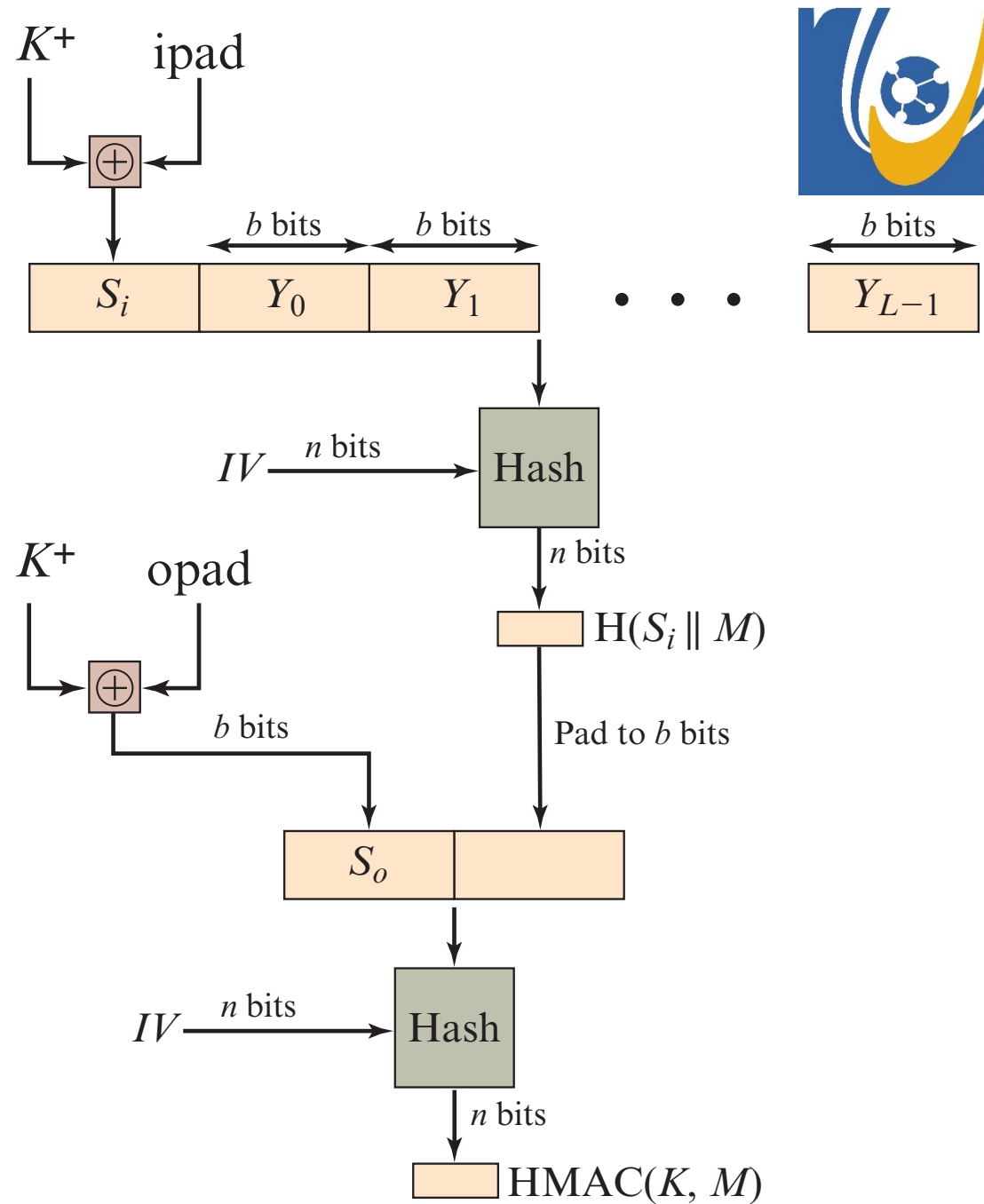
- **MAC based on a hash function**
 - because **hash functions are generally faster**
 - crypto hash function code is widely available
- Hash includes a key along with message
- Original proposal:
$$\textit{KeyedHash} = \textit{Hash}(\textit{Key}|\textit{Message})$$
 - some weaknesses were found with this
- Eventually led to development of HMAC



HMAC

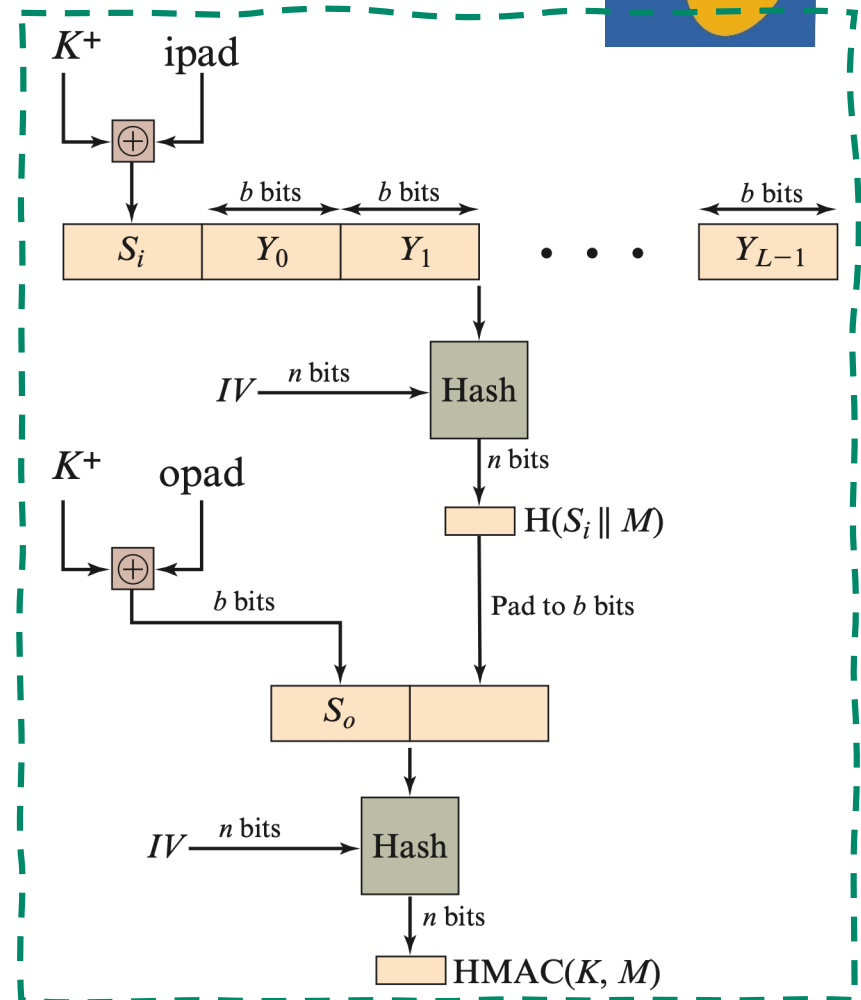
- Specified as Internet standard RFC2104
- Uses hash function on the message:
$$HMAC_K(M) = Hash[(K^+ \oplus opad) || Hash[(K^+ \oplus ipad) || M]]$$
 - K^+ : is the key padded out to size
 - $opad, ipad$: specified padding constants
- Overhead is just 3 more hash calculations than the message needs alone
- Any hash function can be used

HMAC Overview



HMAC Overview

1. Append zeros to the left end of K to create a b -bit string K^+ (e.g., if K is of length 160 bits and $b = 512$, then K will be appended with 44 zero-bytes).
2. XOR (bitwise exclusive-OR) K^+ with $ipad$ to produce the b -bit block S_i .
3. Append M to S_i .
4. Apply H to the stream generated in step 3.
5. XOR K^+ with $opad$ to produce the b -bit block S_o .
6. Append the hash result from step 4 to S_o .
7. Apply H to the stream generated in step 6 and output the result.

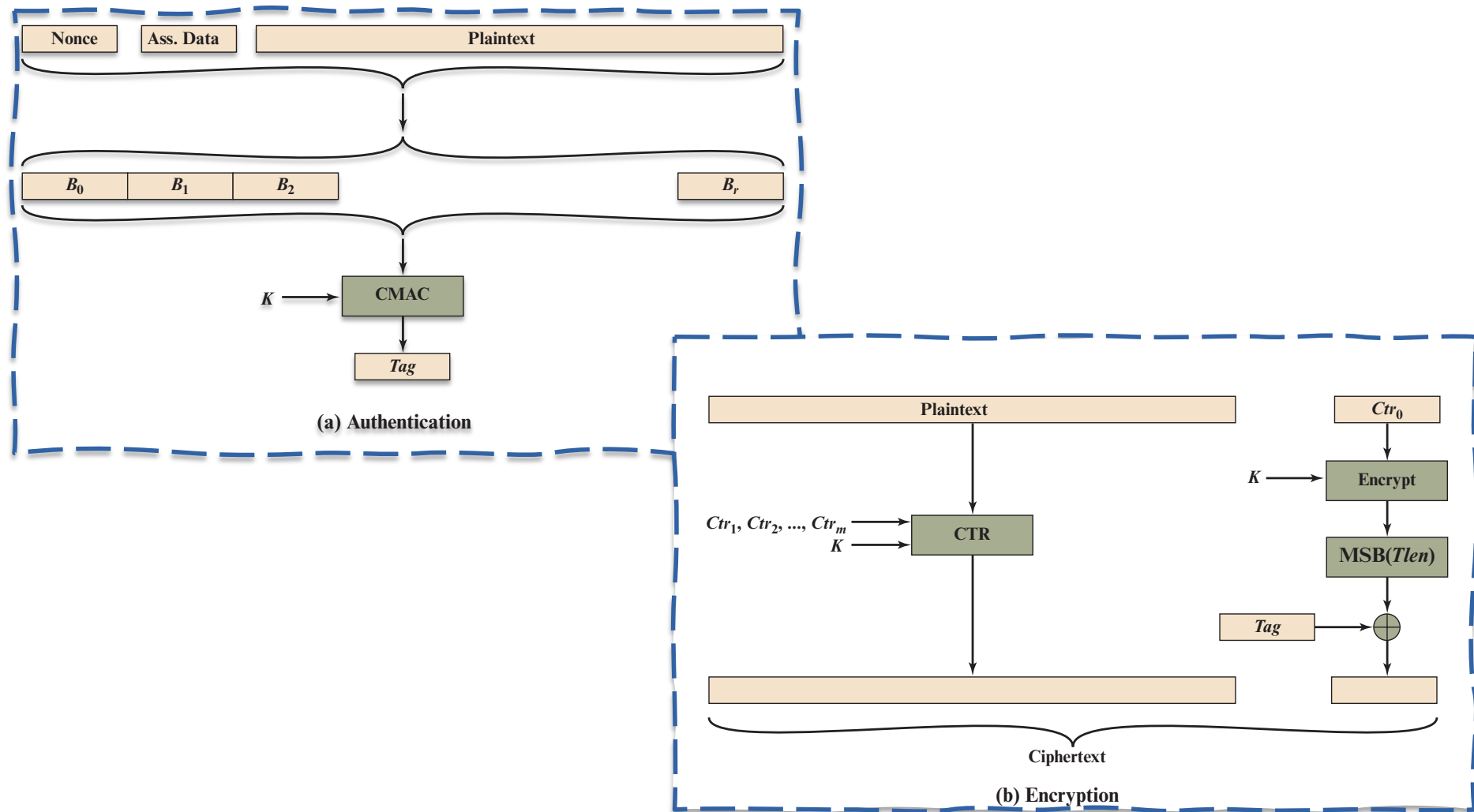




Authenticated Encryption

- Simultaneously protect confidentiality and authenticity of communications
 - Message needs confidentiality and Integrity
- Required to counter active attacks
- Approaches
 - **Hash-then-encrypt:** $E(K, (M || H(M)))$
 - **MAC-then-encrypt:** $E(K_2, (M || MAC(K_1, M)))$
 - **Encrypt-then-MAC:** $(C = E(K_2, M), T = MAC(K_1, C))$
 - **Encrypt-and-MAC:** $(C = E(K_2, M), T = MAC(K_1, M))$

Counter with Cipher Block Chaining-Message Authentication Code (CCM)





Introduction

Unkeyed Hash Functions

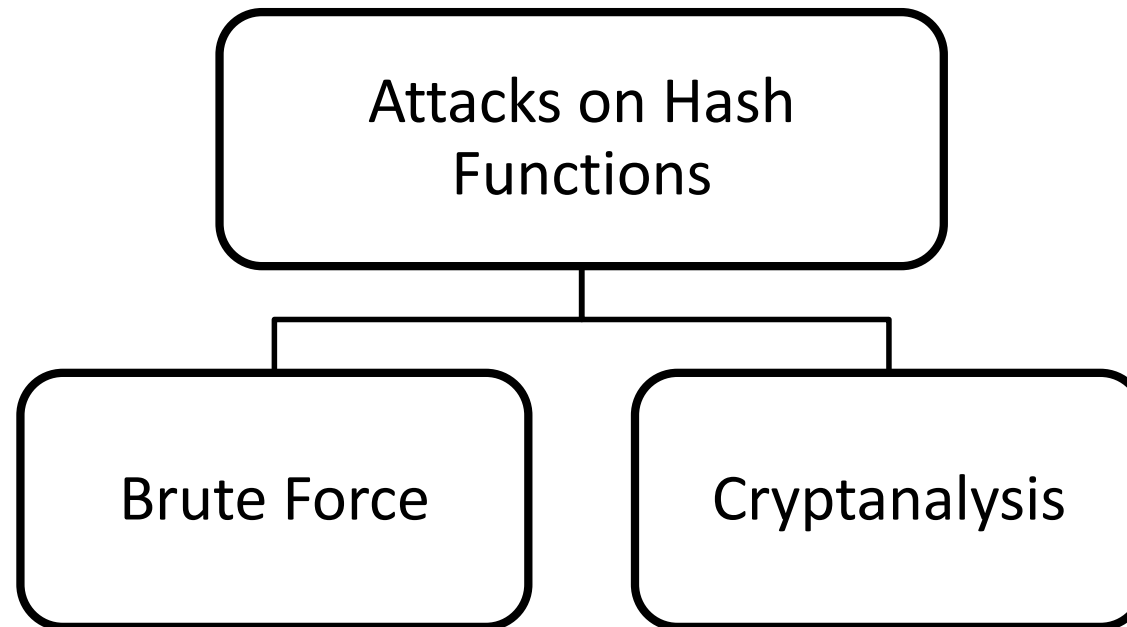
Message Authentication Codes

Attacks on Hash Functions

Key Derivation Functions



Attacks on Hash Functions



Attacks on Hash Functions



- **Second pre-image attack**

- find y such that $H(y)$ equals a given hash value
- Brute force: m -bit hash value $\Rightarrow 2^{m-1}$ times attack and less
- Reference: "Second Preimages on n -bit Hash Functions for Much Less than 2^n Work", Kelsey and Schneier



Attacks on Hash Functions

- **Collision resistance**

- Find x & y with same hash: $H(x) = H(y)$
- m -bit hash value $\Rightarrow 2^{\frac{m}{2}}$ times attack
 - Less effort compared to Brute force attacks
 - Reason: Birthday Paradox

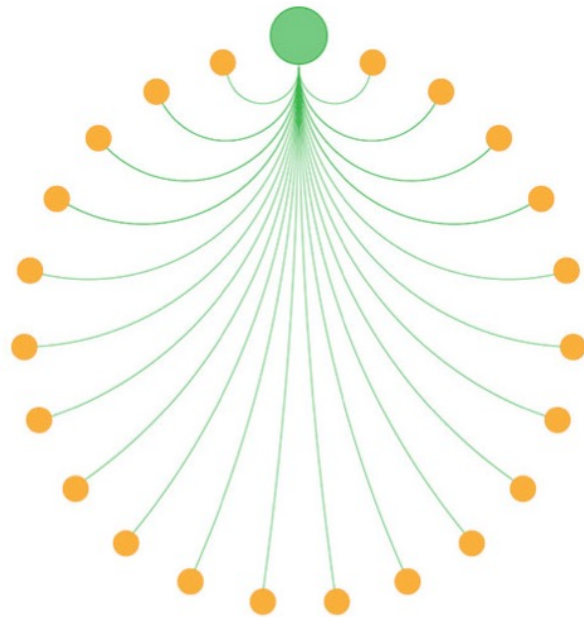


Birthday Paradox

- What is the probability that two people have the same birthday (day and month)?

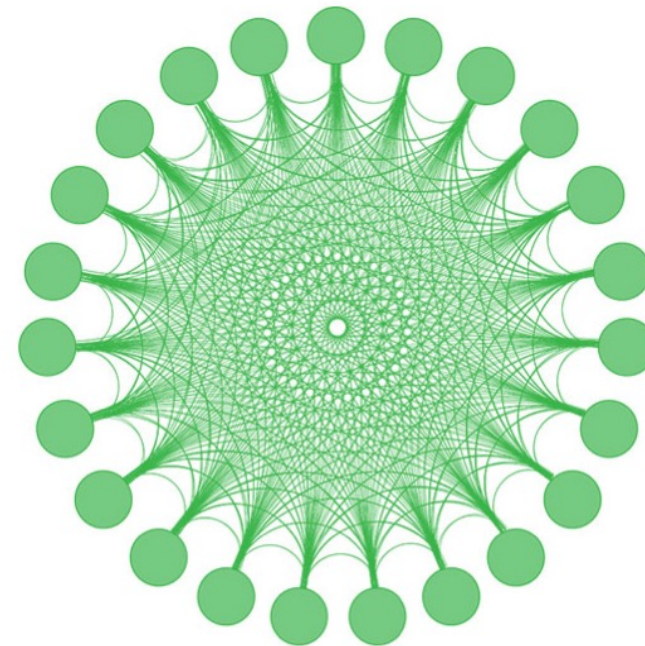


Birthday Paradox



ONE-TO-MANY

The probability of someone sharing your specific birthday is a narrow search so the chances are low. With 23 people there's a 5.9% chance someone shares your birthday.



MANY-TO-MANY

When you are looking for *any* two people to share any birthday the network of possible connections is much richer. With 23 people there's a 50.7% chance two people share a birthday.



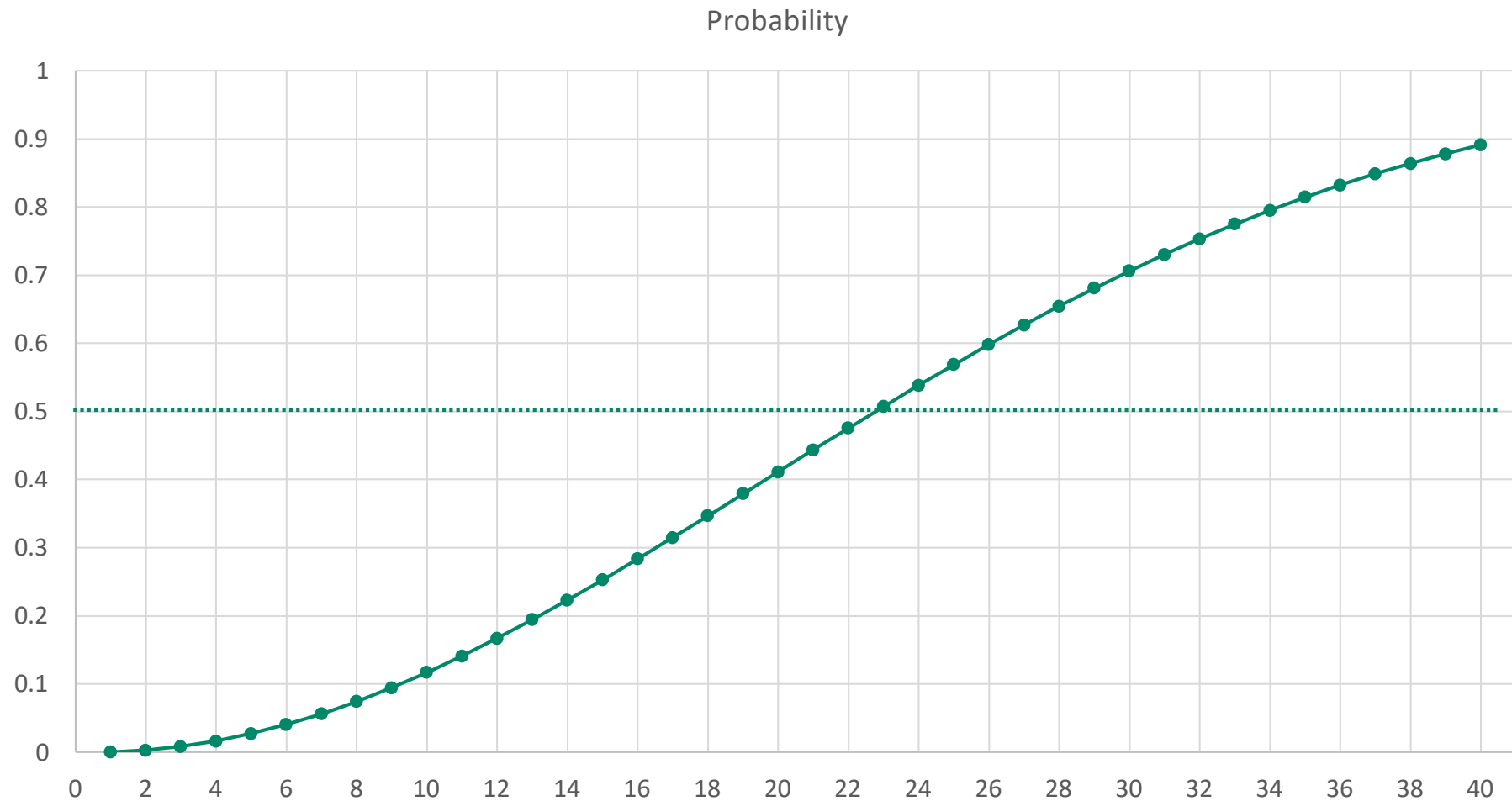
Birthday Paradox (cont'd)

- What is the probability that two people among K people have the same birthday (day and month)?

K	Total	Different Birthday
2	365^2	365×364
3	365^3	$365 \times 364 \times 363$
...		
k	365^k	$365 \times 364 \times \dots \times (365 - k + 1)$

- $P(\text{no common day}) = \frac{[365 \times 364 \times \dots \times (365 - k + 1)]}{365^k} = \frac{365!}{[365^k \times (365 - k)!]}$
- $P(\text{at least one common day}) = 1 - P(\text{no common day})$

Birthday Paradox (cont'd)





Generalization: Collisions in a finite space

- Suppose:
 - N possible values (e.g., birthdays, hash outputs)
- We sample random values
- Question:
 - How many samples before a collision appears?
- Result:
 - Collision expected after $\sqrt{N}$

Birthday Paradox – Application on Hash functions



- Hash functions are pseudorandom
- m bits hash function has 2^m possible output values
- Select n random messages: $m_1, m_2 \dots m_n$ ($n \approx \sqrt{2^m}$), and calculate their hashes, there is more than 50% probability to find a collision
- Examples:

<i>Hash function</i>	<i>Output (bits)</i>	<i>B</i>	<i>n</i>
<i>SHA1</i>	60	2^{160}	2^{80}
<i>SHA – 256</i>	256	2^{256}	2^{128}
<i>SHA – 512</i>	512	2^{512}	2^{256}

Security Implications - Summary



- Collision resistance is weaker than expected
- Hash size = n bits
- Collision security = only $\frac{n}{2}$ bits
- Examples:
 - 128-bit hash $\rightarrow$ 64-bit security
 - 256-bit hash $\rightarrow$ 128-bit security
- Important distinction:
 - Preimage resistance: 2^n
 - Collision resistance: $2^{\frac{n}{2}}$



Introduction

Unkeyed Hash Functions

Message Authentication Codes

Attacks on Hash Functions

Key Derivation Functions



Introduction

- **Problem:** Need to generate many secret keys starting from:
 - **Master Key** or **Source Key:** Generated by a Hardware or a Key Exchange Protocol
 - **Password**



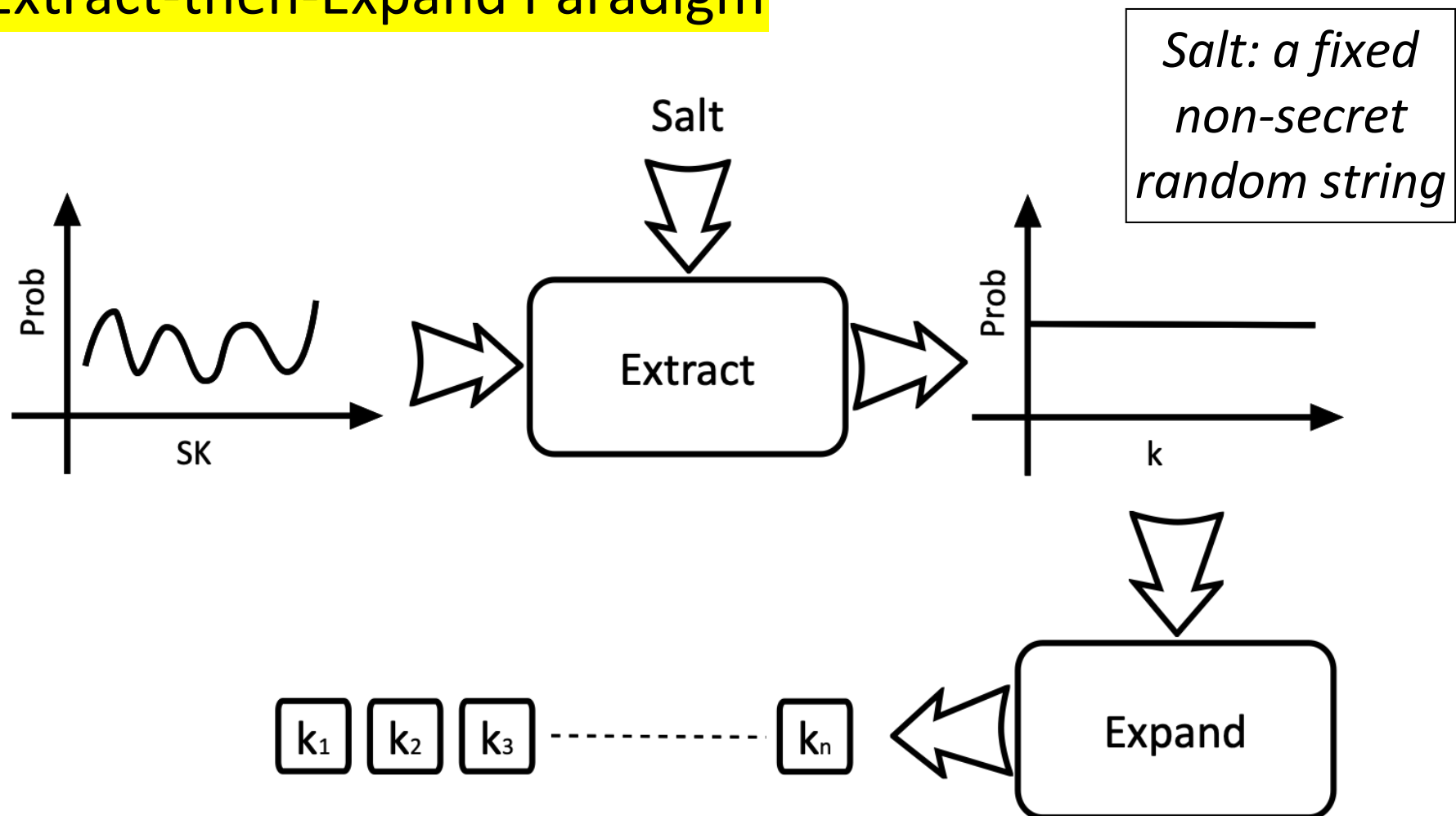
Definition

- A **Key Derivation Function (KDF)** is a **deterministic algorithm** to derive a key of a given size from some secret value.
 - If two parties use the same shared secret value and the same KDF, they should always derive exactly the same key



Key Derivation Function from a Source Key

- **Extract-then-Expand Paradigm**





HKDF: a KDF from HMAC

- [RFC 5869]
- Extract-then-expand paradigm
 - Extraction:
 - Use: $k \leftarrow \text{HMAC}(\text{salt}, SK)$
 - Expansion
 - Using *HMAC* with key k



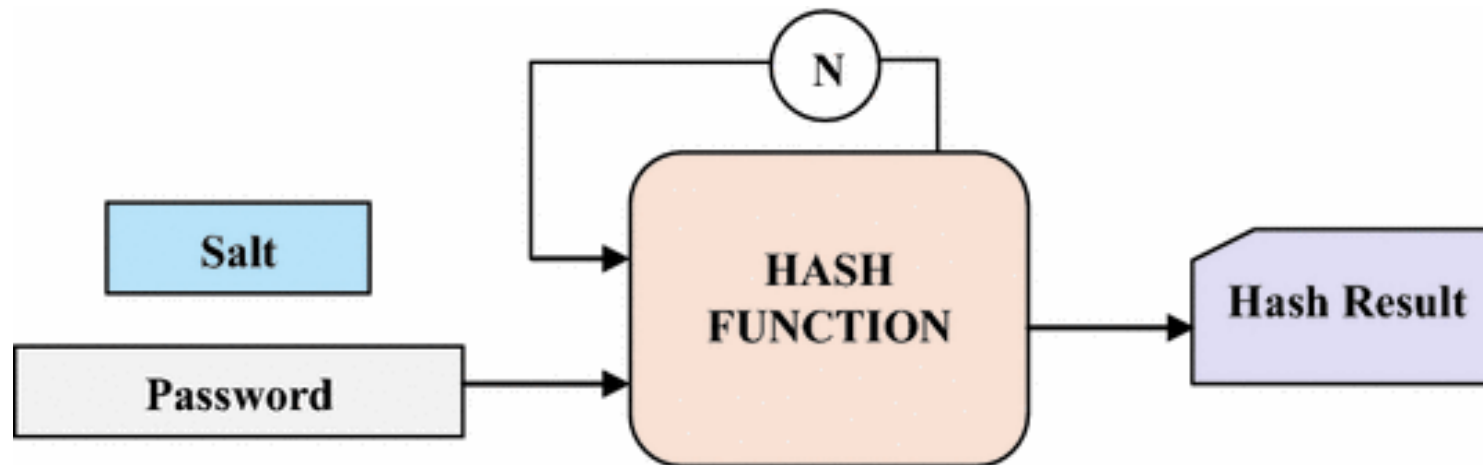
Key Derivation Function from a Password

- Can we use HKDF to derive keys from passwords?
 - HKDF are **not secure** for Passwords
 - Derived keys will be **vulnerable to dictionary attacks**



Key Derivation Function from a Password

- Solution: **Password-Based KDF(PBKDF)**
 - **Salt**
 - **Slow hash function**: iterating hashing MANY times
- Standard approach: PKCS#5 (PBKDF1, PBKDF2)





PBKDF1

- How it works ($dkLen$ is the length of DK)
 - $T_1 = Hash(Pwd \parallel Salt)$
 - $T_2 = Hash(T_1)$
 - ...
 - $T_c = Hash(T_{c-1})$
 - $DK = T_c < 0 \dots dkLen - 1 >$
- Output length depends on the used Hash function (Main Limitation of PBKDF1):
 - MD2 (16 bytes), MD5 (16 bytes), SHA-1 (20 bytes).



PBKDF-2

- Applies a **pseudorandom function to derive keys**.
 - Example: HMAC-SHA1, HMAC-SHA2, etc.
- The length of the derived key is essentially unbounded



PBKDF-2 (cont'd)

- PBKDF2 KDF has five input parameters:

$$DK = PBKDF2(PRF, Password, Salt, c, dkLen)$$

- ***PRF*** is a pseudorandom function of two parameters with output length *hLen* (e.g. a keyed HMAC)
- ***Password*** is the master password from which a derived key is generated
- ***Salt*** is a cryptographic salt
- ***c*** is the number of iterations desired
- ***dkLen*** is the desired length of the derived key
- ***DK*** is the generated derived key



PBKDF-2 (cont'd)

- How it works?
 - $DK = T_1 \parallel T_2 \parallel \dots \parallel T_{\frac{dkLen}{hlen}}$
 - $T_i = F(Password, Salt, C, i)$
 - The function F is the XOR($\oplus$) of C iterations of chained Pseudo Random Functions (e.g., hash function)
- Example: WPA2 uses
 $DK = PBKDF2(HMAC_SHA1, passphrase, ssid, 4096, 256)$



PBKDF-2 (cont'd)

$$DK = T_1 \parallel T_2 \parallel \dots \parallel T_{\frac{dkLen}{hlen}}$$
$$T_i = F(\text{Password}, \text{Salt}, c, i)$$

